

Тема: Основы C++

**1. История C++ **

- **C** (1972 г., Деннис Ритчи) — язык системного программирования.
- **C++** (1983-1985 гг., Бьярне Страуструп) — развитие языка C.
- **Идея:** Добавить к C возможности **ООП** (объектно-ориентированного программирования), сохранив скорость и низкоуровневый контроль.
- **Название:** "C++" — игра слов: ++ это инкремент, то есть "увеличение C".
- **Стандарты:** C++98, C++11 (большое обновление), C++14, C++17, C++20. Мы изучаем основы, общие для всех.

****2. Компиляция **** Это процесс превращения вашего текста программы в исполняемый файл.

- **Исходный код** (файл `.cpp`) — текст программы, понятный человеку.
- **Компилятор** (GCC, Clang, MSVC) — специальная программа-переводчик.
- **Процесс:**
 1. **Препроцессор:** Подключает библиотеки (`#include ...`).
 2. **Компиляция:** Перевод на машинный код (файл `.obj`).
 3. **Компоновка (Linking):** Связывает ваши файлы с библиотеками.
- **Результат:** Исполняемый файл (`.exe` на Windows), который можно запустить.

3. Структура простой программы

```
// 1. Подключение библиотек
#include <iostream> // для ввода/вывода (cout, cin)

// 2. Пространство имен (чтобы не писать std:: перед cout)
using namespace std;

// 3. Главная функция - точка входа в программу
int main() {
    // 4. Тело программы
```

```
cout << "Привет, мир!" << endl; // Вывод на экран

// 5. Возврат 0 - сигнал об успешном завершении
return 0;
}
```

Важно: Все выполнение начинается с `int main()`.

4. Основные типы данных Тип определяет, какие данные хранит переменная и сколько места занимает.

Тип	Размер (примерно)	Хранит	Пример значения
int	4 байта	Целые числа	42 , -10 , 0
float	4 байта	Вещественные числа с плавающей запятой (меньшая точность)	3.14f , -2.5f
double	8 байт	Вещественные числа с двойной точностью (используется чаще)	3.141592 , -0.001
char	1 байт	Один символ	'A' , '\$' , 'g'
bool	1 байт (часто)	Логическое значение	true (1), false (0)

5. Объявление и инициализация переменных Переменная — это именованная ячейка памяти для хранения данных.

```
// Синтаксис: [тип] [имя] = [значение];

// 1. Объявление с инициализацией (лучший способ)
int age = 25;
double price = 99.95;
char grade = 'A';
bool isOnline = true;
```

```
// 2. Объявление, потом присваивание
int count;
count = 10;

// 3. Несколько переменных одного типа
int width = 1920, height = 1080;
```

6. Константы Переменные, которые **нельзя изменить** после создания.

```
// 1. Ключевое слово const (предпочтительный способ)
const double PI = 3.14159;
// PI = 3.14; // ОШИБКА! Изменить константу нельзя

const int MAX_USERS = 1000;

// 2. Директивы препроцессора (старый стиль, виден везде)
#define MIN_USERS 1
// Лучше использовать const - он безопаснее и современнее.
```

Итог:

1. **C++** — мощный и быстрый язык, развившийся из C.
2. **Компиляция** — перевод программы с C++ на машинный язык.
3. **Программа** начинается с `int main() {}`.
4. **Типы данных:** `int` (целые), `double/float` (дробные), `char` (символ), `bool` (логический).
5. **Переменная:** создаётся так: `тип имя = значение;`.
6. **Константа:** переменная с модификатором `const`, которую нельзя менять.

Простая программа-пример:

```
#include <iostream>
using namespace std;

int main() {
    // Объявление переменных
    const double TAX_RATE = 0.20; // Константа
```

```
double salary = 50000.0;      // Переменная
double tax;

// Вычисление
tax = salary * TAX_RATE;

// Вывод результатов
cout << "Зарплата: " << salary << endl;
cout << "Налог (" << TAX_RATE * 100 << "%): " << tax << endl;
cout << "На руки: " << salary - tax << endl;

return 0;
}
```

Обзор темы: Основы C++

Данная тема представляет собой **критически важный вводный модуль** для любого программиста, начинающего работать с C++. Она отвечает на вопросы: «Откуда появился этот язык?», «Как превратить код в программу?» и «Какие инструменты используются для хранения данных?». Это основа основ.

1. История C++: Эволюция, а не революция

- **Ключевая фигура:** Бьёрн Страуструп.
- **Идея (начало 1980-х):** Расширить язык C, добавив возможности **объектно-ориентированного программирования (ООП)** и другие абстракции, не жертвуя производительностью или совместимостью с C.
- **Название:** Изначально "C with Classes" ("Си с классами"). Позже стал C++ (инкремент "C" — идея развития и улучшения).
- **Стандартизация:** Важные вехи — C++98 (первый стандарт), C++11 (революционное обновление, "современный C++"), C++14/17/20/23 — дальнейшее развитие.
- **Суть:** C++ — это **мультипарадигменный** язык, поддерживающий процедурное, ООП, обобщённое и функциональное программирование. Его философия — «доверять программисту», предоставляя как высокоуровневые абстракции, так и низкоуровневый контроль над памятью и железом.

Вывод: История объясняет, почему C++ такой гибкий, мощный, но и сложный. Он наследует эффективность C и обогащает её мощными средствами для построения крупных систем.

2. Компиляция: От текста к исполняемому файлу

Это процесс преобразования человекочитаемого исходного кода (`.cpp` файл) в машинный код (`.exe` , `.out` и т.д.).

Основные этапы:

1. **Препроцессинг:** Обработка директив `#` (вставка заголовочных файлов `#include` , раскрытие макросов `#define` , условная компиляция `#ifdef`).
2. **Компиляция:** Построчный перевод препроцессированного кода на языке C++ в **ассемблерный код**, специфичный для целевого процессора. Проверка **синтаксиса** и части **семантики**.
3. **Ассемблирование:** Преобразование ассемблерного кода в **объектный код** (бинарный, машинный, но ещё не готовый к запуску). Создаются `.obj` / `.o` файлы.
4. **Компоновка (Линковка):** Связывание всех объектных файлов, а также библиотечных функций (например, из Standard Library) в единый **исполняемый файл**. Разрешение адресов функций между файлами.

Альтернатива: Интерпретация (как в Python) — построчное выполнение без создания отдельного файла. C++ — **компилируемый** язык, что даёт высокую скорость выполнения.

Вывод: Понимание компиляции помогает отлаживать ошибки (синтаксиса на этапе 2, компоновки на этапе 4) и осознавать, как код взаимодействует с железом.

3. Структура программы: Скелет кода

Минимальная программа на C++:

```
// 1. Директива включения заголовочного файла (для ввода/вывода)
#include <iostream>
```

```
// 2. Пространство имён (избавляет от std:: перед cout, cin)
using namespace std;

// 3. Главная функция - точка входа в программу. Выполнение начинается с
int main() {
    // 4. Тело функции
    cout << "Hello, World!" << endl; // Оператор вывода

    // 5. Возврат кода успешного завершения в операционную систему
    return 0;
}
```

- `#include` — подключение библиотек.
- `main()` — обязательная функция, с неё всё начинается.
- `{ }` — ограничивают тело функции.
- `return 0;` — завершение функции `main` (и программы).

Вывод: Чёткая структура облегчает чтение и организацию кода.

4. Основные (встроенные) типы данных: "Ячейки" для информации

Тип определяет:

- **Какой смысл** имеют данные (число, символ, истина/ложь).
- **Сколько памяти** они занимают (в байтах).
- **Какие операции** с ними можно выполнять.

Тип	Размер (примерно)	Описание	Пример значений
int	4 байта	Целое число со знаком.	-10 , 0 , 42
float	4 байта	Число с плавающей точкой (одинарной точности).	3.14f , -0.001f

Тип	Размер (примерно)	Описание	Пример значений
double	8 байт	Число с плавающей точкой двойной точности.	3.1415926535 , 2.71828
char	1 байт	Символ или маленькое целое число.	'A' , 'z' , '\$' , 65 (код 'A')
bool	1 байт	Логическое значение (true / false).	true , false

Важно: Размер `int` может зависеть от архитектуры (16/32/64 бит). Для точного размера есть типы `<stdint>` (`int32_t` , `int64_t`).

Вывод: Правильный выбор типа экономит память и предотвращает ошибки (например, переполнение).

5. Объявление и инициализация переменных: Создание "контейнеров"

Переменная — именованная область памяти для хранения данных определённого типа.

```
// 1. Объявление (декларация): "Будет переменная с именем age"
int age;
```

```
// 2. Инициализация (первое присваивание значения):
age = 25;
```

```
// 3. Объявление с инициализацией (рекомендуемый способ):
double price = 99.95;
char grade = 'A';
bool is_valid = true;
```

```
// 4. Современный стиль инициализации в C++ (предпочтительный
int counter{0};          // uniform initialization - защита от C
float temperature{36.6f};
```



Вывод: Всегда **инициализируйте переменные** при объявлении, чтобы избежать неопределённого поведения (мусора в памяти).

6. Константы: Неизменяемые значения

Константа — именованное значение, которое **нельзя изменить** после создания. Это безопасность и читаемость кода.

Способы задания:

```
// 1. Ключевое слово const (основной способ в современном C++
const double PI = 3.1415926535;
const int MAX_USERS = 1000;
// PI = 3.14; // ОШИБКА компиляции! Изменить const нельзя.
```

```
// 2. Ключевое слово constexpr (константа времени КОМПИЛЯЦИИ,
constexpr int ARRAY_SIZE = 256; // Значение известно на этапе
```

```
// 3. Устаревший способ (макрос через препроцессор, не рекомендован)
#define OLD_PI 3.14
// Не имеет области видимости, может привести к сложным ошибкам
```



Вывод: Используйте `const` везде, где значение не должно меняться. Это защищает код от случайных изменений и часто позволяет компилятору делать дополнительные оптимизации.
